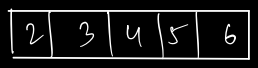
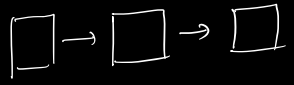
Today's Content :-

1) Trees Intro
2) Naming Convention
3) Tree Traversal
4) Basic Tree Problems

Arrays :



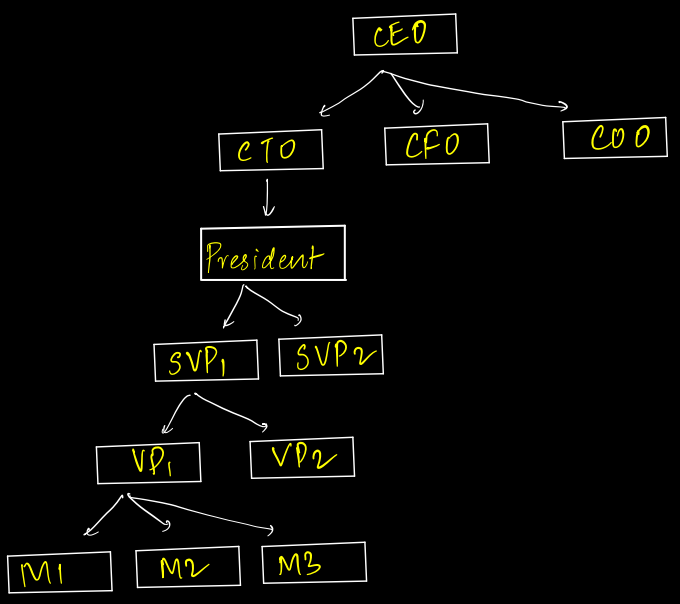
Linked lists

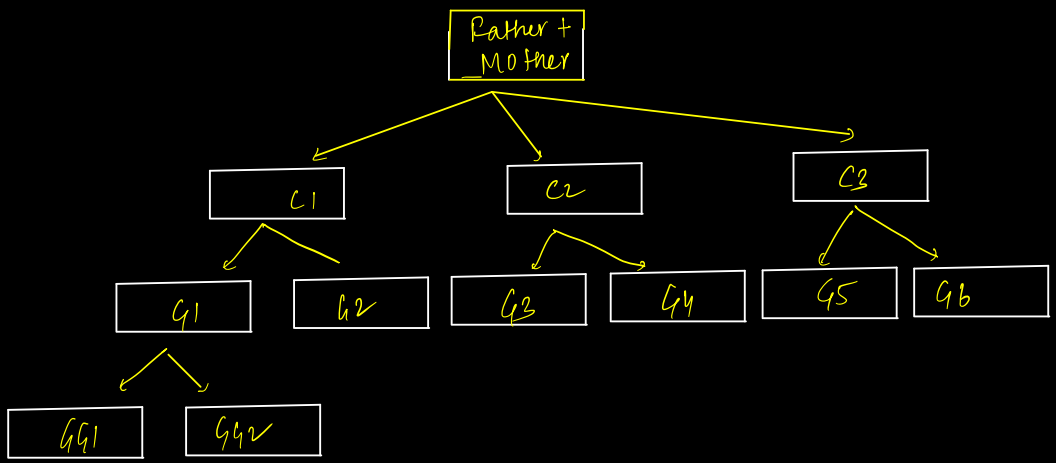


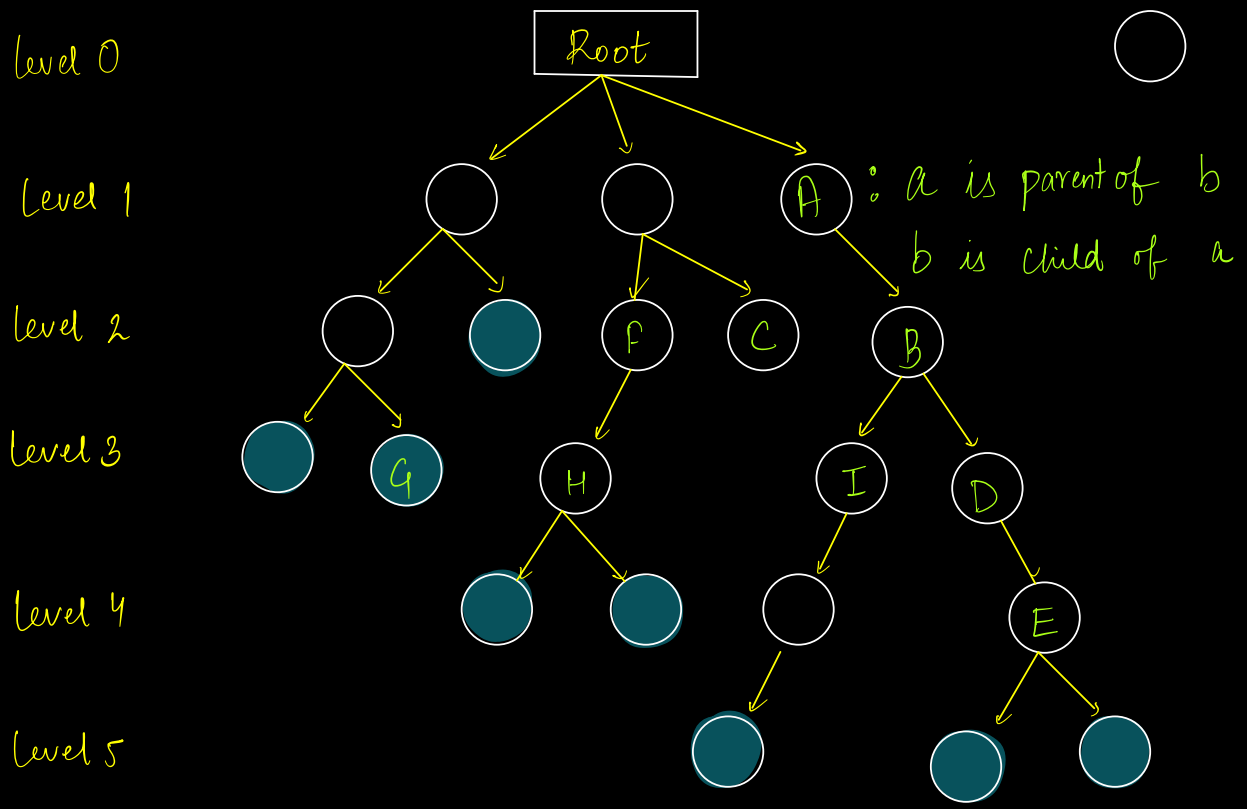
} Linear Data Structures

Heirarchial Data :-

Company Organization :-




Father + Mother
C1
C2
C3
G1
G2
G3
G4
G5
G6
GG1
GG2



A → D: A is ancestor of D & D is descendant of A.

F → C: Siblings { Share same parent }

G, H, I, D: Nodes at same level

root: Node without a parent.

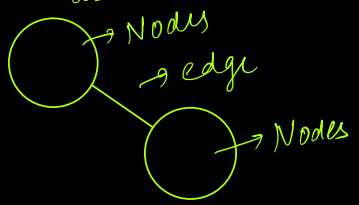
Leaf: Node without children

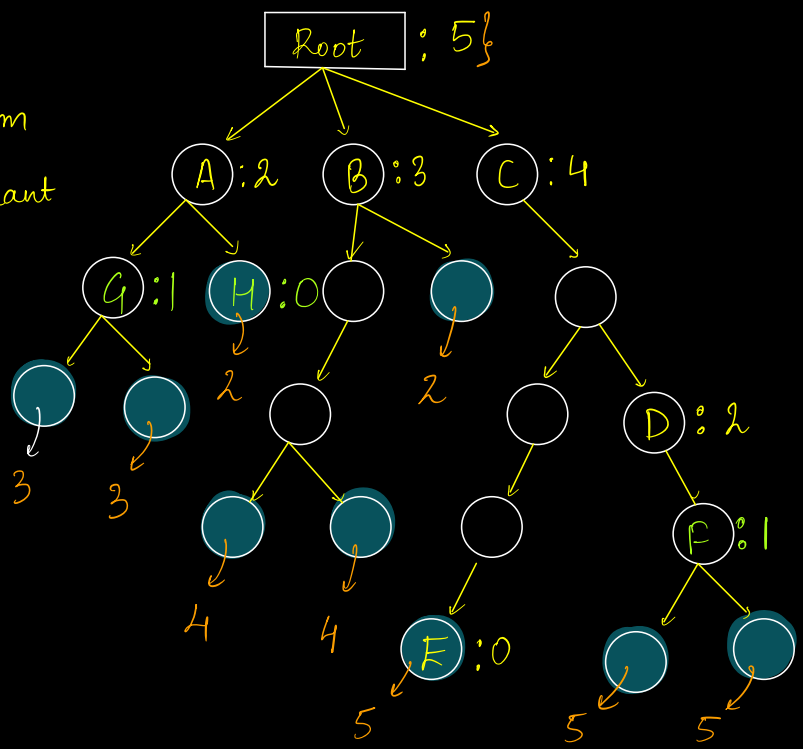
Tree :-
- We will have only 1 root node.
- For every node, only 1 single parent.

Height(Node):

{ Length of longest path from node, to any of its descendant leaf nodes.

Note: Path is calculated based on no of edges





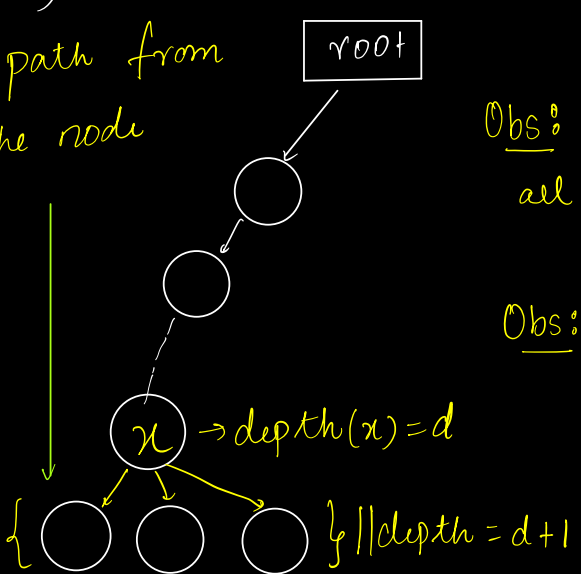
H(A) = 2
H(B) = 3
H(C) = 4
H(D) = 2
H(E) = 0

Obs:
H(Node) = 1 + max(Height of child nodes) — ①

Obs:
H(Leaf node) = 0 — ②

Terminologies:-
Height(tree) = Height(root)
Height(tree) = max depth of any leaf node
Depth(tree)

Depth(Node):-
Length of path from root to the node



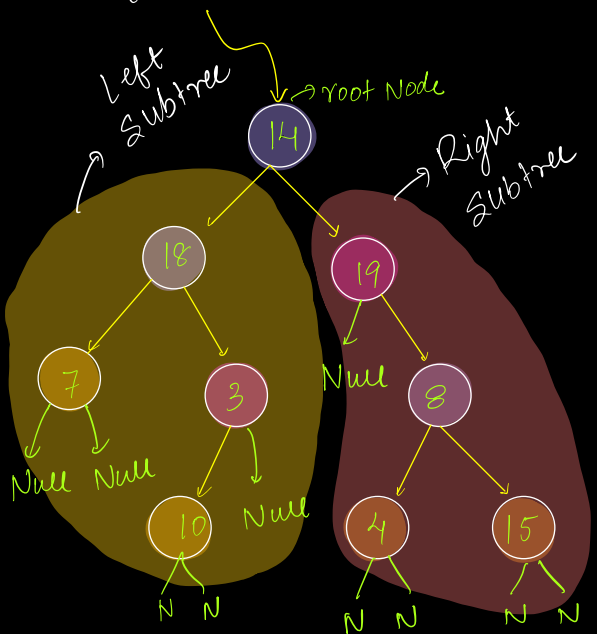
d(A) = 1
d(H) = 2
d(D) = 3

Obs: If depth of node = d
all its child nodes will have
depth = d+1

Obs: depth(root) = 0

{ Height(Node) = Depth(Node) } False

Binary Trees :- Every node can at max have 2 children {0, 1, 2}



→ Leaf Nodes ⇒ 0

→ Nodes with 1 child

→ Nodes with 2 child

root.left is the root of left subtree

root.right is the root of right subtree

```
class Node {
    int data;
    Node left; // obj Reference, which can hold address of node obj
    Node right; // obj reference, which can hold address of node obj
    Node(int x) {
        data = x
        left = NULL
        right = NULL
    }
}
```

```
Node r = new Node(19); // #ad1
```

left	data	right
#ad2	19	#ad3

#ad1

Given root node, you can traverse entire tree

r.left

left	data	right
#ad4	40	#ad5

#ad2

left	data	right
NULL	19	#ad6

#ad3

left	data	right
NULL	19	NULL

#ad4

left	data	right
NULL	19	NULL

#ad5

left	data	right
NULL	19	NULL

#ad6

Going forward,

for all tree problems, tree is always constructed, we are just given root node. }

Tree construction can be explained by Serialization & Deserializations } Adv Module

Tree Traversals:-

- Preorder
- Postorder
- Inorder

} - level order
- vertical order } Adv

Preorder Traversal :-

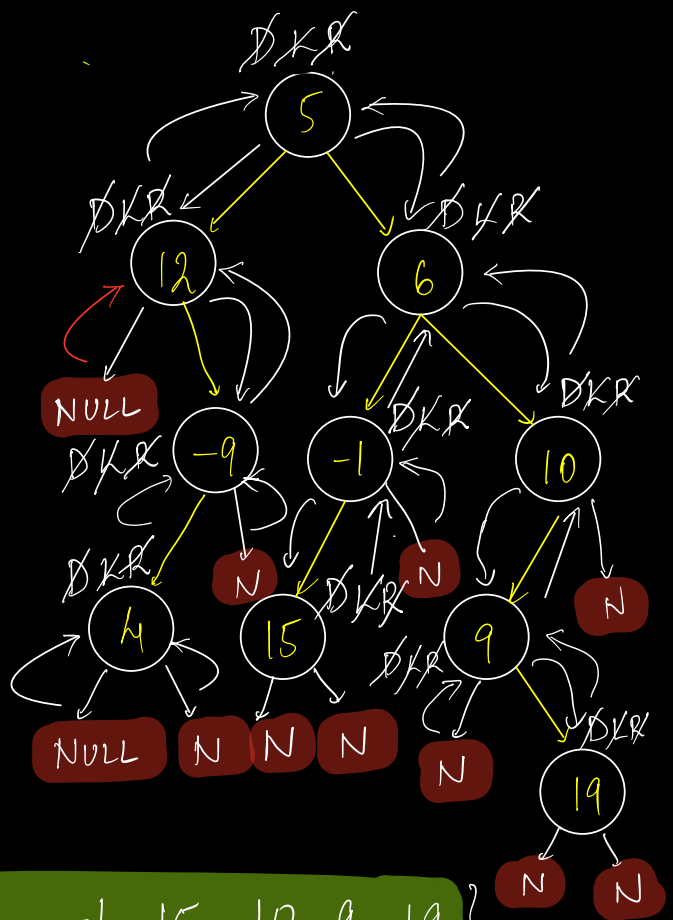
D L R

Data, LST, RST

Step1: print(root.data)

Step2: Go to left subtree & print entire left subtree in preorder

Step3: Go to right subtree & print entire right subtree in preorder

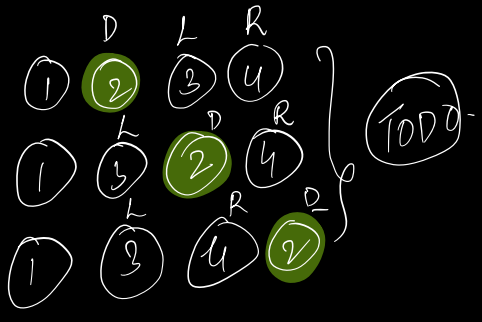


Output : 5 12 -9 4 6 -1 15 10 9 19

Preorder : D L R

Inorder : L D R

Postorder : L R D

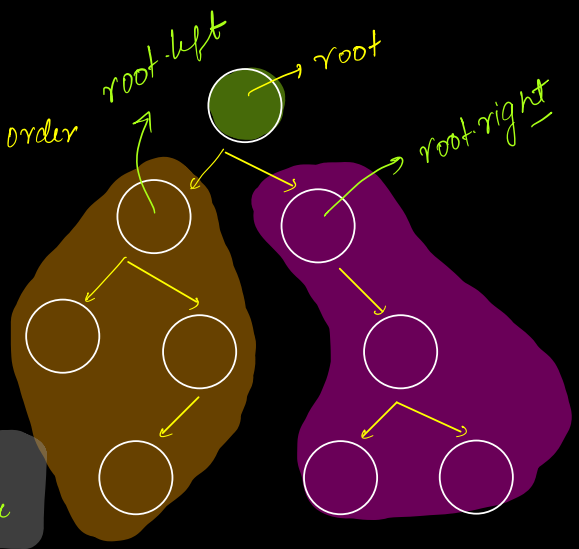


Pseudo Code :-

Ass: Given root node, print entire pre-order

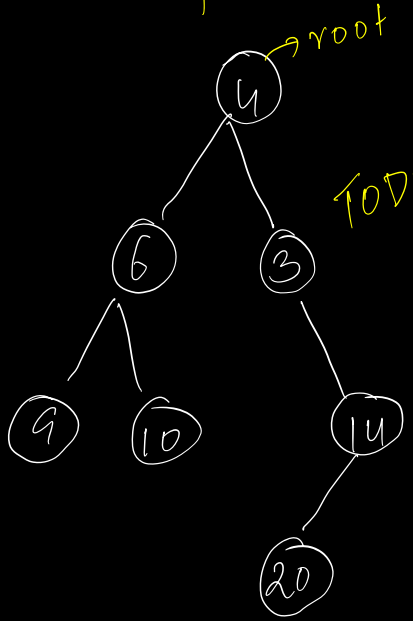
```
void preorder(Node root){
  ① if(root == NULL) return;
  ② print(root.data)
  ③ preorder(root.left);
  ④ preorder(root.right);
}
```

Main Logic



HW

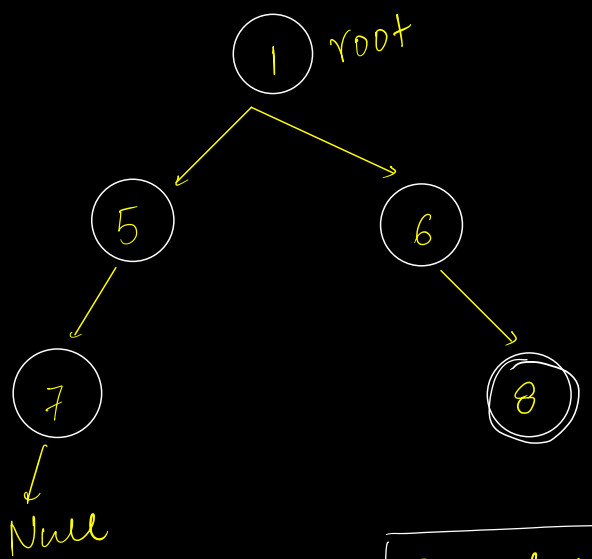
Tree



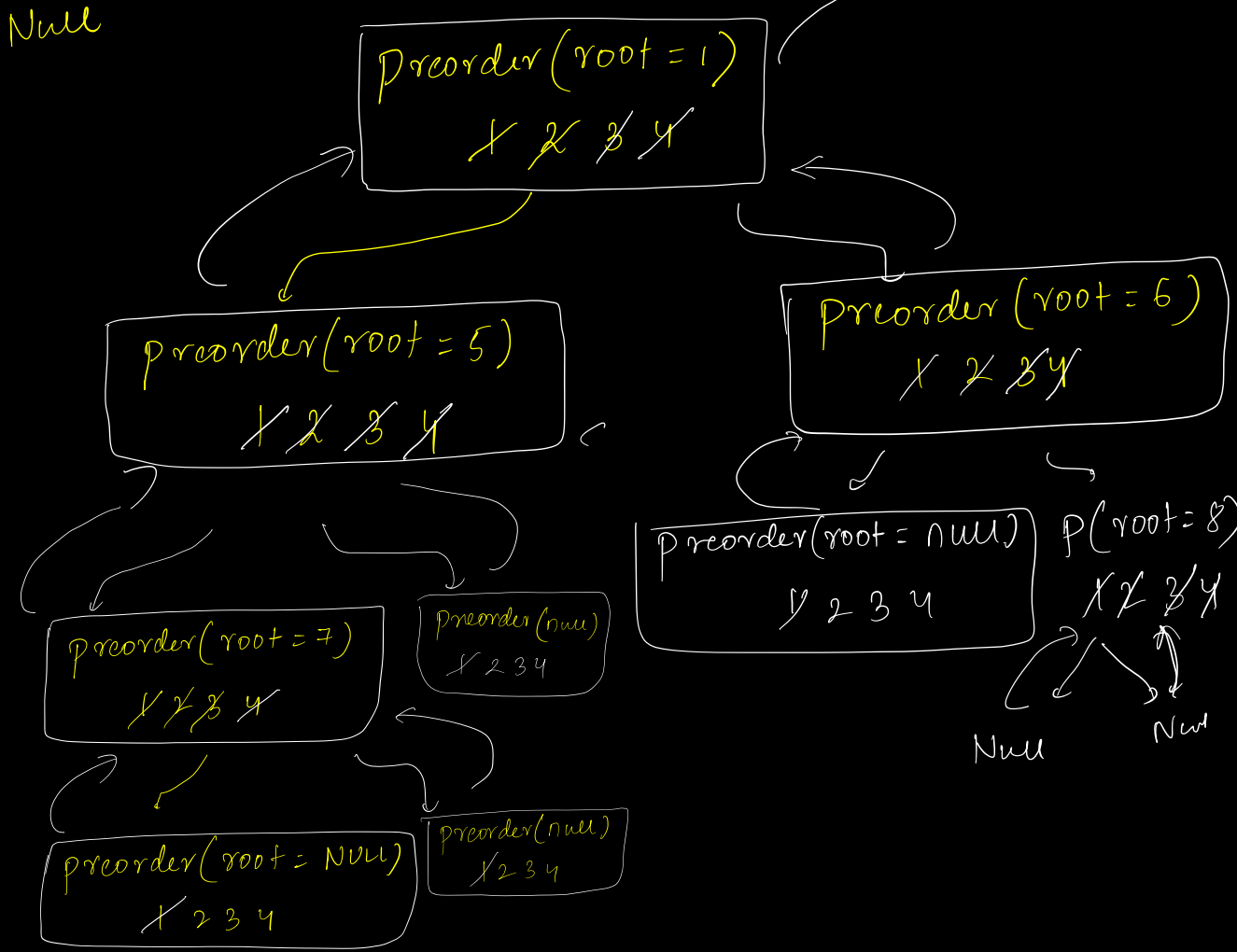
TODO
- Preorder DLR
- Inorder LDR
- Postorder LRD

Preorder : 1, 5, 7, 6, 8

Output : 1, 5, 7, 6, 8

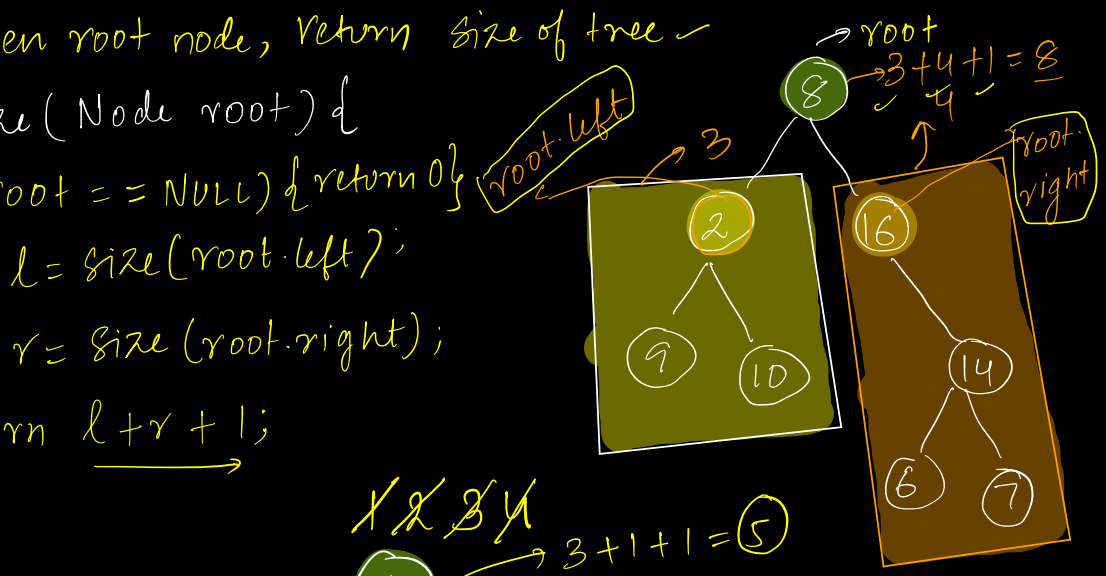


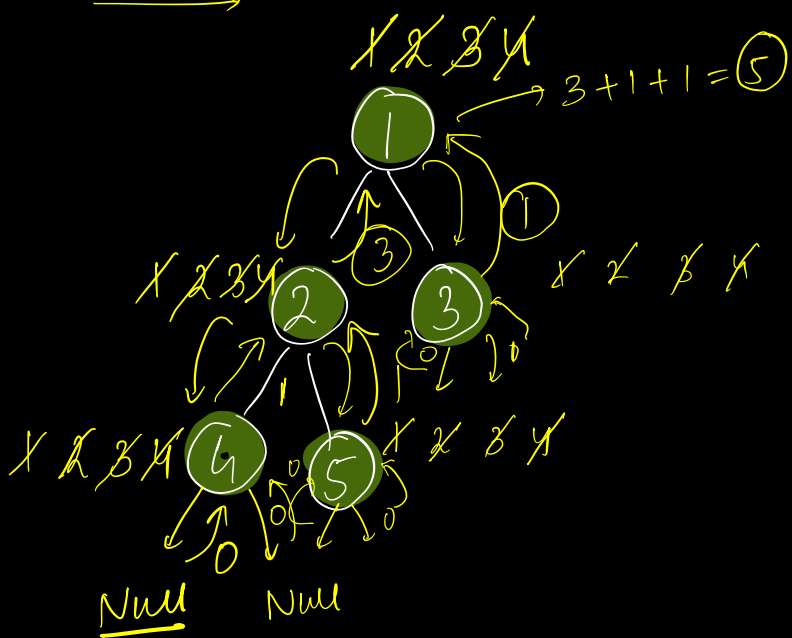

```
Void preorder(Node root){
  ① if(root == NULL) return;
  ② print(root.data)
  ③ preorder(root.left);
  ④ preorder(root.right);
  return;
}
```



Ass: Given root node, return size of tree.

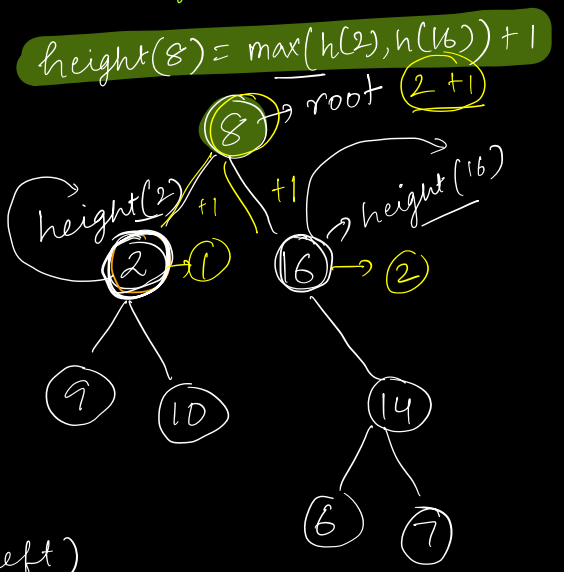
```
int size(Node root) {
① if(root == NULL) { return 0; }
② int l = size(root.left);
③ int r = size(root.right);
④ return l + r + 1;
}
```



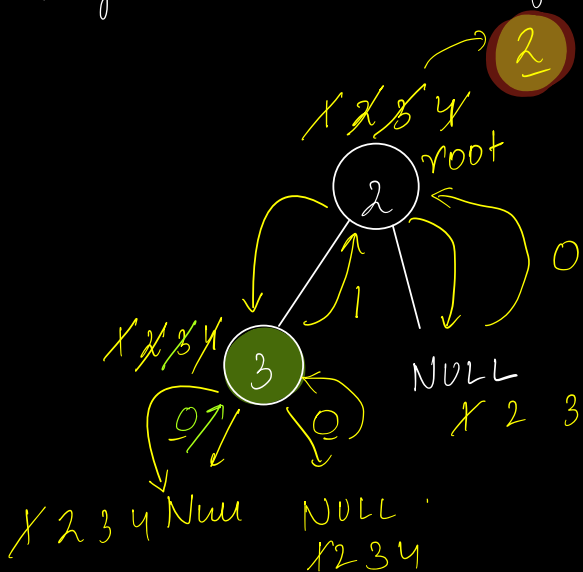


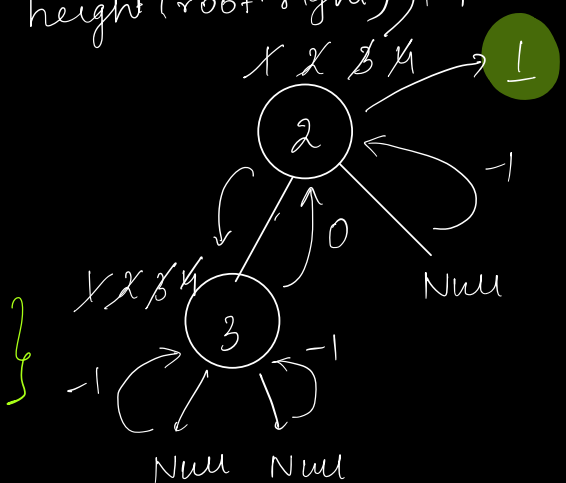
Ass:- Given root node, it will return height of tree

```
int height(Node root) {
① if(root == NULL) return -1;
② int hl = height(root.left);
③ int hr = height(root.right);
④ return max(hl, hr) + 1;
}
```



Height(root) = max(height(root.left), height(root.right)) + 1





Issue: Leaf node, height = 0,
it is returning 1:

Note: In your Assignments, Length of path calculate no of nodes